Guarded Coordination of Untrusted Generative AI Agents in Multi-View Software Workflows

Majid Babaei

Abstract—Multi-agent frameworks built on large language models (LLMs) divide software development among agents that write requirements, designs, code and tests. Every individual answer can look valid while the workflow as a whole still goes wrong. A slow agent may answer a question that has since changed. Two locally valid outputs may disagree on an interface. A reviewer added late may be silently skipped. These are *coordination* failures, and better prompting of each agent does not remove them. We present a coordination semantics in which agents only *propose* primitive values, while a deterministic controller owns artifact structure, dependency tracking and publication. Each generated attribute carries an exact dependency signature over everything it may depend on: its source footprint, the shared context selected for it, and its generation contract. A guarded transition system admits a proposal only if it answers the current question. It invalidates exactly the work affected by a change, bounds generation attempts per dependency episode, and publishes only globally consistent snapshots. We prove structural integrity, currentness of accepted values, exact repair locality, a generation bound, a terminal success-or-failure outcome for acyclic workflows, a forward simulation from asynchronous to atomic execution, and safety of transactional reconfiguration. Counterexamples mark the boundary of each result. We instantiate the semantics in a TLA$^+$ model with scoped machine-checked lemmas and in a Python reference controller. In 3,300 randomized adversarial executions, the reference controller exposed no unsafe value, while every ablation that removes a non-redundant mechanism violated its targeted property. On 32-document workspaces, exact signatures reduced generation requests per change 11.9$\times$ relative to full regeneration and 8.1$\times$ relative to a whole-workspace footprint, without changing any accepted value.

Index Terms—Large language models, multi-agent systems, software engineering workflows, runtime enforcement, incremental consistency, formal semantics, TLA$^+$, model transformation

1 Introduction

Large language models (LLMs) are now routinely embedded in software-engineering workflows [1]. Frameworks such as MetaGPT [2], ChatDev [3] and AutoGen [4] organize several LLM agents into roles (product manager, architect, programmer, tester) that hand artifacts to one another. Agentic tools such as SWE-agent [5] edit real repositories. Recent surveys identify such LLM-based multi-agent systems as a central direction for automated software engineering [6], [7].

Most quality work in this area targets the *individual answer*: is the generated code correct [8], [9], does it parse [10], [11], does it pass a guardrail [12]? Multi-agent systems, however, also fail *between* answers. An empirical study of multi-agent LLM systems attributes many failures to inter-agent misalignment and to missing or incorrect verification of task completion [13]. Consider three situations, each of which a per-answer check cannot detect:

- *Stale answer.* A coding agent is asked to implement requirement revision 1. While it works, the requirement changes to revision 2. The old answer arrives later, parses and passes its tests, and is accepted. It answers a question that no longer exists.
- *Local validity, global conflict.* The design agent declares document identifiers as strings; the test agent independently assumes integers. Each output passes its own validator, yet the combination is inconsistent.
- *Silent reconfiguration.* A security reviewer is added after the workflow looks complete. Unless completion is recomputed, the earlier "done" verdict is reported for a team that has not yet reviewed anything.

These are coordination failures. They concern which inputs an answer was produced from, who was allowed to write which artifact, and whether a collection of artifacts is ready to be released. They arise even when every agent behaves as well as a model can.

Approach. This paper treats agents as *untrusted proposers*. A deterministic controller owns structure, dependencies, acceptance and publication; agents only propose primitive values such as text, code or JSON for slots that the controller has created. For every generated slot, the controller records an exact *dependency signature*: an injective encoding of everything the slot may depend on, namely its source footprint, the shared context selected for it, and the contract (prompt, schema, validator) under which it is generated. A proposal is committed only if, atomically at commit time, it answers the request that is still current, comes from the authorized owner, passes its validator and leaves the artifact structure well formed. A change invalidates exactly the slots whose observation changed. A per-episode budget bounds generation attempts. A publication barrier releases only snapshots in which nothing is dirty, nothing is pending and all joint constraints hold.

Contributions. This paper makes the following contributions.

• *M. Babaei is with the School of Computing, University of the Fraser Valley, BC, Canada.*
E-mail: majid.babaei@ufv.ca

1) *A coordination semantics for untrusted generative proposals* (Sections 3 and 4). It combines a deterministic structural kernel with exact dependency signatures, a per-job control automaton and guarded atomic transitions. Its key distinction is between a value that was *accepted*, one that is currently *usable*, and a snapshot that is *publishable*.
2) *Guarantees with explicit assumptions and counterexamples* (Section 5). We prove structural integrity, current-contract acceptance, rejection of obsolete replies, exact repair locality, a generation bound, terminal outcomes for acyclic workflows, a forward simulation from asynchronous to atomic execution, and safe reconfiguration. Counterexamples show which assumption each result needs.
3) *An executable instantiation* (Section 6). It consists of a TLA$^+$ protocol model with scoped TLAPS lemmas, a Python reference controller with an independent trace checker, and a document-service case study. The case study includes a restricted artifact language for which validator soundness is itself proved.
4) *An empirical evaluation* (Section 7) organized around four research questions. It combines bounded model checking with mutants, an implementation-level ablation under 3,300 randomized adversarial executions, a comparison of invalidation strategies on workspaces of up to 32 documents, 28 change scenarios, and an account of what is and is not mechanized.

All artifacts, raw logs and the scripts that generate every number in this paper are available (Section 11).

Scope. Our guarantees concern *coordination* and are relative to declared validators. We never assume that an agent reasons correctly, and we need no probabilistic model of an LLM: for safety, the proposal source may return anything. Conversely, we do not claim that generated software is functionally correct beyond what its validators check. Section 6.2 shows how a sufficiently restricted artifact language makes that stronger claim provable. The central inductive invariant and the refinement mapping are proved on paper; their full mechanization is ongoing work, and Table 5 states precisely which parts are machine-checked.

Organization. Section 2 introduces the running example and the requirements. Section 3 gives an overview of the approach, and Section 4 the formal semantics. Section 5 states and proves the guarantees. Section 6 describes the instantiation, Section 7 the evaluation, and Section 8 threats to validity. Section 9 discusses related work and Section 10 limitations, and Section 11 concludes.

2 Motivating Example and Problem Statement

2.1 A Multi-Agent Document-Service Workflow

Our running example is a small document service developed by four agents, each responsible for one *view* of the system (Fig. 1). An *intake* view holds service requests and their clauses. A *design* view holds one endpoint per accepted request. An *implementation* view holds one code patch per endpoint. A *verification* view holds one check per clause. A shared *security context* contains findings, such as a path-traversal report, that are relevant to some patches and checks.

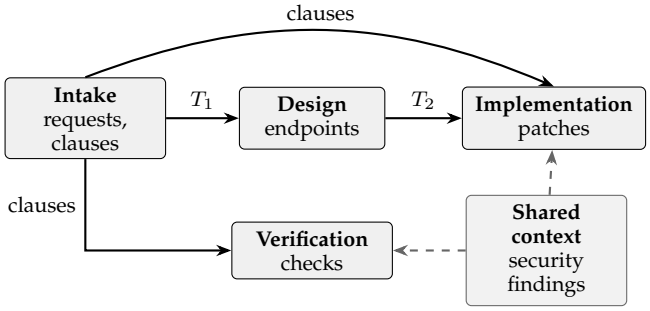


Fig. 1. Running example. Solid arrows are declared source dependencies; T_1 derives an endpoint per request and T_2 a patch per endpoint. Dashed arrows carry selected shared context. The controller creates objects and references; agents propose only primitive attribute values (signature text, patch body, check oracle).

The views are *heterogeneous*: a requirement is neither an operation signature nor a test oracle. Nevertheless, an edit in one view can invalidate artifacts in others. Suppose the clause "an archived document cannot be overwritten" is tightened to "overwriting an archived document returns HTTP status 409 (Conflict)". The endpoint's signature `put(id, body)` does not change. The patch must still be regenerated, because the behavior it must implement changed. A dependency analysis that looks only at the interface would miss this.

2.2 Coordination Failures

The workflow above admits four failure classes that per-answer checks cannot exclude.

- **F1** *Stale reply.* A request is sent while the clause has revision 1, the clause changes to revision 2, and the old reply then arrives. It is syntactically valid and may even pass the old tests.
- **F2** *Local validity, global inconsistency.* The design agent chooses string identifiers and the verification agent integer identifiers. Both outputs pass their local validators.
- **F3** *Unsafe change of team or rules.* A reviewer is added, a worker's permission is revoked, or a prompt or validator changes. Old evidence is then reused, or old completion verdicts are kept, as if nothing had changed.
- **F4** *Unbounded or wasted generation.* A failing request is retried on every scheduler pass. Conversely, an unrelated edit causes every artifact to be regenerated.

2.3 Requirements

A coordinator for untrusted generative agents should satisfy the following requirements. Section 5 answers each with a theorem.

- **R1** *Structural integrity.* Artifact objects, traces and references are well typed and have exactly one producer, whatever the agents return.

R2 *Currentness.* A value is usable only if it was validated against the *current* inputs, contract and owner. Obsolete and duplicate replies cannot write (F1, F3).

R3 *Honest completion.* Publication requires that no work is pending or dirty and that all joint constraints hold (F2, F3).

R4 *Locality.* A change invalidates exactly the work whose inputs changed, not more and not less (F4).

R5 *Bounded effort and termination.* Generation attempts are bounded, and a stable workflow ends in an explicit success or failure verdict (F4).

R6 *Safe reconfiguration.* Changing agents, rules, permissions or validators preserves R1–R3 (F3).

3 Approach Overview

3.1 Design Principles

The approach rests on four principles.

1) *Agents propose, the controller disposes.* Agents never create objects, references or completion verdicts. They return a primitive value for a slot that the controller created, and the controller decides what happens to it.
2) *Depend on exactly what was observed.* Each slot's request is built from an explicit *observation*: the source data, selected context and contract that the agent may see. Its exact encoding, the signature, is the slot's dependency key.
3) *Commit atomically and only if current.* The check "is this reply still the answer to the current question?" and the write are a single transaction.
4) *Separate accepted, usable and publishable.* An accepted value may become outdated after an edit. It remains as history, but consumers may read only *usable* values. Release happens only through a global *publication barrier*.

3.2 Walk-Through of One Change

Fig. 2 shows the architecture. We follow the tightened 409 clause through it; Fig. 3 shows the same run as a message sequence, including a stale reply.

① An authorized editor changes the clause. Every input change and every configuration change is one atomic controller transaction.

② The *structural normalizer* recomputes matches of the transformation rules ("one endpoint per accepted request", "one patch per endpoint"). It creates or deletes target objects and their traces. Here no object changes, because the request still exists.

③ For each generated slot, called a *job*, the controller recomputes the observation, meaning the exact data the agent will see. The patch job's observation contains the endpoint signature *and* the clause text, so its signature changes. An unrelated request's jobs keep their signatures and their accepted values remain usable.

④ A job whose signature changed starts a new *episode* with a fresh attempt budget, and any request still in flight for the old episode is cancelled. Jobs whose prerequisites are unusable are suspended rather than dispatched.

TABLE 1
Key concepts and notation

Concept	Sym.	Meaning
View	–	A typed model graph (e.g., the design model) with stable object identities.
Rule, match	r, q	A transformation rule and one match of it; q identifies the target object it creates.
Job	j	A pair (q, b): one generated primitive attribute b of the target of match q, e.g., a patch body.
Footprint	F_j	The source data a job's agent may read.
Observation	o_j	Footprint, selected context and contract identity η_j: everything the job depends on.
Signature	σ_j	Exact injective encoding $\mathrm{enc}(o_j)$; the job's dependency key.
Stamp	a	The signature against which the current accepted value was validated.
Episode	e	A maximal period with one signature; carries an attempt counter $n \leq k$.
Request	u	One dispatched generation call, identified by (j, e, n) and never reused.
Usable	U_j	Accepted, stamp equals current signature, and all prerequisites usable.
Publishable	Pub	Normal form, nothing suspended, dirty or pending, and global predicate G true.
Contract	η_j	Identity of instruction, schema, validator, environment and generation settings.

⑤ The controller dispatches a request with a fresh identity $u = (j, e, n)$ (job, episode, attempt). The agent receives only the observation and the static instruction.

⑥ The agent returns a proposal, perhaps late, perhaps twice, perhaps malformed.

⑦ The *commit guard* checks, inside one transaction, that u is still active, that the signature recorded in u equals the job's current signature, that the owner is still authorized, that the validator accepts the value, and that the resulting structure normalizes. Only then does the value become *usable*. The stale reply in Fig. 3 fails the first two checks and is discarded.

⑧ The *publication barrier* releases a snapshot only if no required stage is suspended, no job is dirty or pending, and the global predicate G holds. For example, G requires that design and verification agree on the identifier type (F2). Publication ⑨ installs the snapshot with its revision.

3.3 Key Concepts

Table 1 defines the vocabulary used in the rest of the paper. Plain-language meanings come first; the formal objects follow in Section 4.

4 Formal Semantics

We model the controller as a labeled transition system $(\mathcal{S}, \mathcal{S}_0, \Lambda, \rightarrow)$ with states $\mathcal{S}$, initial states $\mathcal{S}_0$, event labels Λ and a transition relation. An *invariant* holds initially and is preserved by every transition. Signatures, identities and primitive values are unbounded, so the system is infinite-state. Diagrams such as Fig. 4 are therefore automaton *schemas*, not claims that the workflow is a finite-state machine. For safety, the proposal source is fully nondeterministic: it may return any value, malformed data, duplicates,

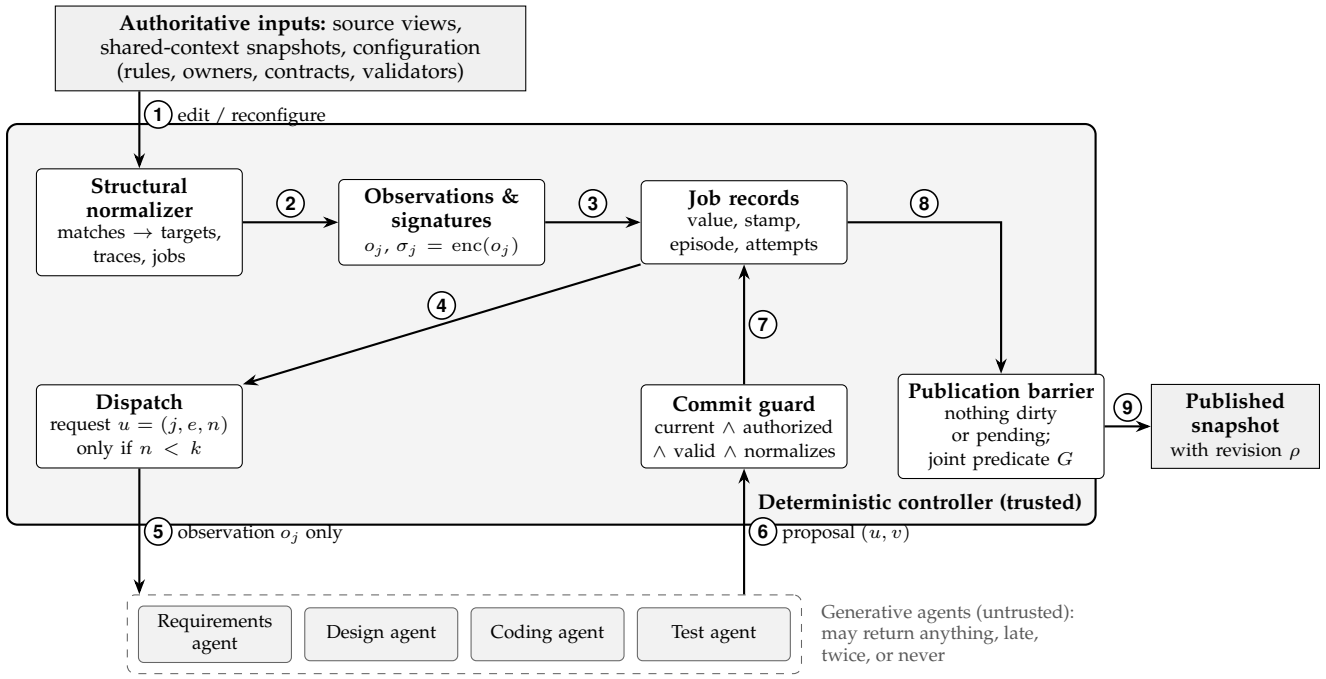


Fig. 2. Architecture of guarded coordination. Agents (bottom) receive only a declared observation and return a primitive proposal. Everything else, including structure, dependency signatures, attempt budgets, acceptance and publication, belongs to the deterministic controller. Numbered steps are explained in Section 3.2: (1) input change, (2) structural normalization, (3) signature comparison, (4) dirty job with budget left, (5) dispatch, (6) proposal, (7) guarded commit or rejection, (8) usability check, (9) publication.

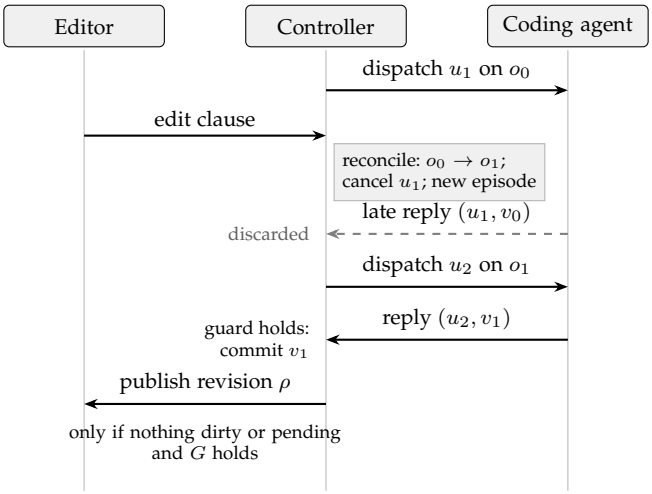


Fig. 3. Failure F1 handled by the controller. The edit cancels request u_1; its late reply cannot commit even if it passes the old validator. If the clause later reverts to o_0, a new episode starts and u_1 remains cancelled.

or nothing. This over-approximates every possible LLM behavior and needs no probability assumptions.

4.1 Views, Rules and the Structural Kernel

A *view* is a finite graph with stable object identities, typed primitive attributes (strings, Booleans, numbers or inert code text) and typed references. A metamodel fixes types, multiplicities and well-formedness predicates. Let X denote the authoritative source views together with immutable shared-context snapshots. Collection membership, object existence and the absence of a context item are observable data: a dependency analysis that looked only at existing attribute values would miss additions and deletions.

A configuration C contains view metamodels, transformation rules, ownership permissions, context-selection policies, validators, generation contracts and a global predicate G. Reads are *ranked* (assumption A0 in Table 3): rule matches, structural expressions, observations, selectors and validators read only external data or strictly earlier stages. A *stage* groups independent rules processed after their prerequisites.

A rule has a tuple source pattern, a deterministic guard, one target object, deterministic structural bindings, and zero or more generated primitive bindings. A match is identified by the key $q = (r, \mathrm{id}(s_1), \ldots, \mathrm{id}(s_m))$, such as $(\mathsf{RequestToEndpoint}, \mathsf{request17})$. A deterministic injective constructor yields the target identity $\iota(q)$ and the persistent trace $q \mapsto \iota(q)$. Producers use disjoint identifier namespaces. A reference resolves through traces only if the referenced match exists; a missing mandatory target makes normalization fail rather than leaving a dangling reference.

The *structural normalizer* N_C maps a coherent view snapshot to an internal skeleton. It computes the current match set, creates missing targets, removes obsolete targets and their traces, and computes deterministic attributes and references. It keeps accepted generated values only for surviving bindings whose contract remains compatible. Generated slots that have no value yet hold $\perp$. N_C is partial: inadmissible candidate snapshots are rejected.

Definition 1 (Structural admissibility)**.** Configuration C is *structurally admissible* on a snapshot if N_C terminates, is deterministic and idempotent, produces exactly one owned target and trace per match key, creates only well-typed ref-

erences, and satisfies all multiplicity constraints. Generated bindings may write only declared primitive slots, and every write has exactly one configured owner.

Admissibility is a proof obligation for a concrete transformation engine, not a property of arbitrary transformation programs. A simple sufficient construction allocates all targets first and resolves references second, a separation familiar from ATL [14].

Theorem 1 (Constructive structural kernel)**.** *Consider finite source graphs and finitely many single-target rules whose guards and structural expressions are pure and terminating. Suppose target identities $\iota(q)$ are injective, producer namespaces are disjoint, primitive expressions have declared types, and each reference expression returns a finite set of current match keys of the required target type and multiplicity. Then enumerating matches, allocating all targets, and resolving references constructs an admissible skeleton. The result is unique, and idempotent when generated slots are held fixed.*

Proof. Each source product is finite, so terminating guards yield a finite match set. Allocation creates one target per key; injectivity and disjoint namespaces exclude collisions, and the trace is exactly the graph of ι on that set. Each reference key lies in this domain by hypothesis, so second-phase lookup succeeds and preserves type and cardinality. Typed primitive expressions preserve attribute types. Purity makes the match set and structural values independent of enumeration order, which gives uniqueness. Repeating the construction yields the same keys, identities, references and values, which gives idempotence. Deleting targets outside the current key set yields exact coverage, and all references are recomputed in the same transaction. Induction over the finite rank order extends the argument to all enabled stages. □

If a prerequisite stage is unavailable, its dependent stage is *suspended*: it contributes no active targets, traces or jobs, and old accepted values are kept only in a private cache that consumers cannot read. A set Q records suspended required stages, so an empty job set is never mistaken for completion. On reactivation, stable keys reconstruct the objects, and cached values are restored only if observation, contract and authority are compatible.

4.2 Observations and Dependency Signatures

A job $j = (q, b)$ is one generated attribute b of the target of match q. Let Z be the coherent readable snapshot formed from X and the currently usable generated predecessors, built in rank order. The job's *effective observation* is

$$o_j = \mathrm{obs}_{C,j}(Z) = \big(F_j(Z),\ \mathrm{select}_{C,j}(Z),\ \eta_{C,j}\big), \qquad (1)$$

where F_j is a pure, total source-footprint expression, $\mathrm{select}_{C,j}$ returns the authorized context items (including explicit absence markers), and $\eta_{C,j}$ identifies the generation and validation contract: instruction, schema, validator and its environment, source expression, selection policy and generation settings. A configuration change need not invalidate every job, but it must change η for every job it affects.

The *signature* $\sigma_j = \mathrm{enc}(o_j)$ is an injective canonical encoding, so different observations never share an encoding. In the mathematics it is exact equality. An implementation may store the encoding or use a digest with collision detection; a bare cryptographic digest makes the results conditional on the absence of collisions. Context version numbers enter o_j only if the agent or validator actually sees them. Otherwise, equal selected content at different versions has equal signatures, and no regeneration is needed (scenario S10 in Section 7.5).

A footprint describes a dependency *function*, not merely the locations visited last time. The query "all findings for endpoint e" depends on the membership of that set as well as on its contents, so adding a matching finding must change the observation (scenario S09). Unavailable or access-denied context yields $o_j = \bot$ and blocks dispatch; silently substituting empty text would change the contract.

Definition 2 (Footprint adequacy, validator soundness)**.** Let $R_j(Z, v)$ be the desired domain relation between snapshots and proposed values. A footprint is *adequate* for R_j if there is $\widehat{R}_j$ with $R_j(Z, v) = \widehat{R}_j(o_j, v)$ for all admissible Z, v. A validator $V_j(o, v)$ is *sound* for R_j if $V_j(o, v) \Rightarrow \widehat{R}_j(o, v)$.

Informally, two snapshots with the same observation must yield the same validity judgment for any proposal. Adequacy is a semantic obligation; signature equality is a mechanical test. A footprint containing only the endpoint signature is *not* adequate for the 409 clause, because two snapshots may have equal signatures and different clauses. No freshness mechanism can repair an inadequate footprint.

4.3 Controller State and the Job Automaton

The controller state is $S = (C, X, Y, L, B, P, A, H, Q, \nu, \rho)$: configuration, authoritative inputs, active skeleton, traces, job records, active requests, immutable history and evidence, published snapshot, suspended required stages, a monotone nonce allocator and a semantic revision counter. A job record holds an accepted triple (v, a, γ) (value, stamp, evidence pointer) or none, a failed signature f or none, an episode nonce e, the episode signature s, and an attempt counter $n \in \{0, \dots, k\}$ for a fixed budget $k \geq 1$. An episode is a maximal period during which an active job keeps one signature. Returning to an earlier signature, or reactivating a suspended job, starts a *new* episode. Nonces are allocated monotonically, so a request identity $u = (j, e, n)$ is never reused, even after deletion and recreation of the job. The history A retains every dispatched request, which lets late and duplicate deliveries be recognized.

For an active job, freshness, usability and the set of outstanding obligations are defined in rank order:

$$\mathrm{fresh}_j(S) \iff B_j \text{ holds } (v, a, \gamma) \ \wedge\ a = \sigma_j(S), \qquad (2)$$

$$U_j(S) \iff j \text{ active} \wedge \mathrm{fresh}_j(S) \wedge \textstyle\bigwedge_{i \prec j} U_i(S), \qquad (3)$$

$$O(S) = \{j \in \mathcal{J}(S) : \neg U_j(S)\}. \qquad (4)$$

Here $i \prec j$ ranges over declared generated prerequisites. Downstream consumers read a value only through the U_j gate. If an upstream value becomes unusable, all its descendants become unusable too, even if their old bytes are still stored. Validators are total and deterministic on their declared observations; validators that call external tools must include tool versions in the contract.

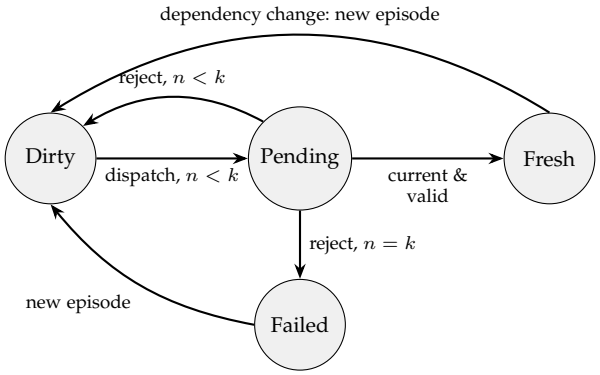


Fig. 4. Control automaton schema of one job. A dependency change cancels pending authority and starts a new episode (shown here from Fresh). Compatible cached evidence can restore Fresh directly. Obsolete replies never cause a transition.

Fig. 4 summarizes a job's control states, which are derived from the record. A dependency change cancels pending authority and starts a new episode. The job is then Dirty, unless compatible cached evidence immediately makes it Fresh again (for example, after reversion to an earlier observation). Failed is terminal for an unchanged episode but is not a successful state. No agent can move a job directly to Fresh.

4.4 Transition Rules

Every transition is one atomic controller transaction; Table 2 lists them. Two rules deserve comment.

Resolve. For a returned pair (u, v), let σ_u and o_u be the signature and observation recorded in request u. The commit guard requires that u is active in P, that it belongs to the same job and episode, that its owner is still authorized, that all prerequisites are usable, that the value has the slot's primitive type, and that

$$\sigma_u = \sigma_j(S), \qquad o_j(S) \neq \bot, \qquad V_j(o_u, v). \tag{5}$$

Because a generated attribute may influence later matches or references, the value is first installed in a private *candidate* state, on which complete downstream normalization and reconciliation run. The candidate is committed only if every enabled structural check succeeds; otherwise the attempt counts as failed. The equality test and the write must share one linearization point [15] with edits and reconfigurations. Validation may run outside the transaction on the immutable o_u, but its result counts only after the guard is rechecked inside the transaction. This is optimistic concurrency control [16].

Publish. Let G be a total predicate over an immutable candidate snapshot that encodes all joint constraints and required executable checks. Then

$$\begin{aligned}\mathrm{Publishable}(S) \iff & \text{ normal form} \wedge Q = \emptyset \wedge O(S) = \emptyset \\ & \wedge P = \emptyset \wedge G(C, X, Y, B, L).\end{aligned} \tag{6}$$

Only a Publish transition satisfying (6) installs H, with that exact snapshot and revision. Later edits leave H as a historical snapshot but remove any claim that it represents the current state.

4.5 Assumptions

Table 3 collects the assumptions used by the guarantees. They are independent: disjoint ownership protects writes, exact signatures protect dependency comparison, and sound validation connects acceptance to a domain property. None implies another.

5 Guarantees

This section states and proves the guarantees, organized by the requirements of Section 2. Table 5 at the end summarizes each guarantee with its assumptions and supporting evidence.

5.1 Safety (R1–R3)

A committed value need not remain fresh after an edit; the controller must instead prevent its *use* until it is repaired. We therefore distinguish a historical acceptance invariant from current usability. Claiming that every stored value is always fresh would be false in exactly the change scenarios of interest.

Theorem 2 (Controller safety)**.** *Under A0–A4, from a normalized initial state with no accepted values, every reachable state satisfies: (i) active targets and traces have unique producers and valid structural references; (ii) each accepted record carries evidence of an authorized successful validation against its recorded observation and contract; (iii) every usable value satisfies the current validator for the current observation; (iv) every committed reply was current and authorized at its commit linearization point; and (v) every installed published snapshot satisfied* (6) *at installation.*

Proof. By induction on run length. Initially, (i) holds by A1 and the other claims are vacuous. *Edit* preserves (i) by A1. It keeps acceptance evidence only as history and recomputes usability, so a changed observation cannot make an old value usable unless observation and contract agree exactly. *Dispatch* changes no accepted value or structure. A successful *Resolve* writes the owned slot in a candidate and normalizes the dependent structure before admitting it; Theorem 1 or A1 gives (i) for the new graph, and a rejected candidate changes nothing. Request identity and authorization give the provenance part of (ii); the validator and exact-signature tests give (iii) and (iv). Failed and discarded replies write no accepted value, and timeouts change only attempt state. *Publish* tests the exact snapshot it installs, which gives (v). *Reconfigure* normalizes under the new admissible configuration, invalidates incompatible evidence and cancels obsolete authority, so it preserves (i)–(iv) and never relabels a historical publication as current. In (iii), equality of encoded observations implies equality of validator inputs and contract by A2 and A4. □

Corollary 3 (Validator-relative domain correctness)**.** *If a usable value v has an adequate footprint and a sound validator (Definition 2), then $R_j(Z, v)$ holds. If G is sound for a desired snapshot relation R_G, every published snapshot satisfies R_G.*

Proof. Theorem 2(iii) gives $V_j(o_j, v)$; soundness gives $\widehat{R}_j(o_j, v)$; adequacy identifies it with $R_j(Z, v)$. The second claim applies soundness of G to the publication guard. □

TABLE 2
Controller transitions (concrete machine) and their counterparts in the atomic specification used for refinement (Theorem 4). Unmentioned components are unchanged; $\mathcal{R}$ is rank-order normalization and reconciliation.

Transition	Guard	Effect	Abstract action
Edit / Normalize	Authorized edit; candidate passes N_C	Install input; apply $\mathcal{R}$: new episodes for changed signatures, cancel their requests, suspend unavailable stages; advance ρ. On failure keep state, record rejection.	Edit
Dispatch(j)	$j \in O(S)$; $o_j \neq \bot$; prerequisites usable; no active request; $f \neq \sigma_j$; $n < k$	$n \leftarrow n + 1$; record request (u, j, e, σ_j, o_j) and owner in P and A; send only o_j and static instruction	Attempt (budget increment)
Resolve (commit)	$u \in P$; same job/episode; owner authorized; Eq. (5); type check; candidate normalizes	Install (v, σ_u, γ); clear f; remove u; apply $\mathcal{R}$; advance ρ	Accept
Resolve (reject) / Timeout	$u \in P$ current, but validation, typing or normalization fails, or timeout	Remove u; if $n = k$ set $f \leftarrow \sigma_j$	Fail
Discard	$u \notin P$, obsolete or unauthorized	None (duplicate or late delivery)	Stutter
Publish	Eq. (6) on an immutable snapshot	Install H with that snapshot and revision	Publish (without $P = \emptyset$)
Reconfigure	Candidate C' admissible, ownership and policy compatible, satisfies A0 (and A5 if progress is required)	Install C'; apply $\mathcal{R}$; migrate compatible records; cancel requests whose job, authority or contract changed; advance ρ	Reconfigure

TABLE 3
Assumptions and the guarantees that require them

ID	Assumption	Used for
A0	Finite ranked read dependencies; unavailable stages are suspended	Well-defined semantics
A1	Admissible deterministic normalizer; agents write primitives only	Structure
A2	Exact effective signatures; every relevant contract change alters η	Currentness, locality
A3	Linearizable controller transactions; immutable observations	Commit safety
A4	Total validators over recorded observations; stable declared environment	Acceptance
A5	Finite acyclic dependency graph, single producers, stage stability	Termination
A6	Inputs and configuration stabilize; fair controller; requests settle	Progress
A7	Adequate footprints; sound local and global validators	Domain correctness

Example 1 (Limits of parsing and test evidence)**.** A patch that returns 200 for every request may parse and type-check while violating the 409 clause; such a validator is not sound for that clause. A generated test that fails on one empty stub need not detect an incorrect implementation. Locally accepted endpoints with different identifier types can violate a joint interface constraint, which G or a certified cross-binding validator must check. Rejection at the barrier is an honest blocked outcome, not a promise that automatic repair will succeed.

5.2 Asynchronous Refinement (R2)

Theorem 2 is stated for the concrete, asynchronous controller. We now show that asynchrony adds no behavior beyond an atomic specification. The abstract machine $\mathcal{A}$ has the same configuration, inputs, structure, accepted records, attempt and failure records, and publication, but no remote requests. Its actions are those in the last column of Table 2. *Accept* may choose any value that satisfies the current observation's validator and ownership condition, provided the candidate normalizes. Let $\alpha(S)$ erase requests and delivery detail while retaining semantic and budget records. Note that concrete dispatch maps to an abstract budget increment, not to a stutter: claiming that dispatch stutters while an incremented counter remains visible would be incorrect.

Theorem 4 (Forward simulation)**.** *Under A0–A4, every concrete run projects through α to a run of $\mathcal{A}$, allowing stuttering. Hence asynchronous replies cannot introduce a value forbidden by the atomic specification.*

Proof. We map each concrete transition. Dispatch maps to Attempt, since both increment the same episode budget and request creation is erased. A successful Resolve maps to Accept: the concrete signature test implies that the recorded observation is exactly the current abstract observation, the remaining tests imply the abstract validator and ownership guards, and both use the same candidate normalization. A current rejection or timeout maps to Fail, including the exhaustion mark. Obsolete and duplicate deliveries change no retained record and map to stutters; signature reconciliation belongs to the associated Edit or Reconfigure in both machines. Edit, Reconfigure and Publish use the same normalized snapshots, and concrete Publish has all abstract guards. Initial projections agree, so induction yields a projected run. We do not claim the converse or any liveness correspondence [17], [18]. □

The guards are not redundant in general. If both cancellation of obsolete requests and the commit-time signature check are removed, the sequence "dispatch on o_0, edit to o_1, deliver the old valid answer" commits obsolete provenance. Either defense alone suffices in the one-job model. A check-then-write race outside the transaction bypasses both. Without unique request identities, a duplicated reply can overwrite a newer accepted value after deletion and recreation. Section 7.3 confirms each of these mechanisms experimentally.

5.3 Locality and Isolation (R4)

For two normalized snapshots with the same job identities and contract, the *direct impact set* is

$$D(Z, Z') = \{j : o_j(Z) \neq o_j(Z')\}. \tag{7}$$

Newly created jobs are dirty and deleted jobs are removed; structural matching is handled separately.

Theorem 5 (Exact direct invalidation)**.** *Assume all surviving jobs are usable before one authorized edit, no job is pending, and no generated value changes during the edit transaction. For each surviving job whose prerequisites remain available, its accepted stamp becomes stale if and only if $j \in D(Z, Z')$. Jobs outside D retain their validator-relative value. Descendants of unusable prerequisites are additionally blocked until those prerequisites are repaired.*

Proof. Before the edit, $a_j = \mathrm{enc}(o_j(Z))$; after it the stored value is unchanged and the current signature is $\mathrm{enc}(o_j(Z'))$. Injectivity gives $a_j \neq \sigma_j(Z')$ exactly when $o_j(Z) \neq o_j(Z')$. If the observations agree, validator input and contract agree, so the earlier validation still applies. The prerequisite gate in (3) explains descendant blocking. □

An unchanged upstream output permits *early cutoff*. After an upstream job is repaired to the same value, an observation-equal descendant becomes usable again without regeneration (scenario S16). This is the main source of savings in RQ2. The dependency graph must cover guards, membership queries, structural expressions, validators and selected context as well as prompt text; a graph that omits any of these supports neither locality nor scheduling.

Proposition 6 (Observation-relative isolation)**.** *If two states agree on a job's observation, contract, attempt record, request activity, authorization and prerequisite usability, they admit the same dispatch eligibility and the same set of locally valid proposals for that job, even if unselected source or context content differs.*

Proof. Every local guard is a function of the listed data, so substituting equals preserves its truth and the validator's accepted-value set. □

This is a statement about the controller's inputs, not information-theoretic confidentiality: pretraining, logs or tool access may reveal more unless they are controlled separately.

5.4 Bounded Generation (R5)

Let E_j be the number of signature episodes of job j during a finite interval, counting episodes interrupted by edits, started by reactivation, by a change of η, or by deletion and recreation.

Theorem 7 (Generation bound)**.** *For any finite execution interval that starts with fresh episode budgets,*

$$N_{\mathrm{dispatch}} \leq k \sum_j E_j. \tag{8}$$

If each request u has at most p_u input tokens and a provider-enforced output cap ℓ_u, the accounted generation tokens satisfy $T \leq \sum_u (p_u + \ell_u)$, which is at most $k(p+\ell)\sum_j E_j$ for common caps p, ℓ.

Algorithm 1 Topological repair of a stable revision

```
Require: admissible C, frozen external revision X, budget k ≥ 1
for stage s in topological order do
    if a required predecessor of s is failed or blocked then
        classify s as blocked; continue
    normalize s; create/delete jobs; reconcile signatures
    for each job j in s with ¬U_j do
        while ¬U_j ∧ n_j < k ∧ o_j ≠ ⊥ do
            guarded Dispatch(j)
            await reply or timeout
            guarded Resolve
        if ¬U_j then classify j as failed
if Publishable(S) then
    publish the snapshot
else
    report failure, blockage or global conflict
```

Proof. Within an episode n starts at zero, dispatch strictly increments it, and no transition decreases it, so the guard $n < k$ allows at most k dispatches. Retries, timeouts and obsolete replies do not reset n within the episode. Summing over episodes gives (8); summing the per-request caps, including those of discarded requests, gives the token bound. □

The bound charges observed episodes, not distinct signature values: the input sequence a, b, a, b earns four budgets, so continuous editing can cause unbounded work. The simpler bound $k|D|$ holds when each affected job is processed once after its prerequisites stabilize. The theorem bounds generation calls and tokens; it does not bound matching, validation or wall-clock time. A request to "be brief" is not an enforced cap.

5.5 Terminal Outcomes (R5)

Acyclicity must hold for all data dependencies, not merely for arrows between agent names. Assumption A5 partitions jobs and structural stages into a finite directed acyclic graph. Jobs in one stage are mutually independent, each node has one producer, and no output influences its own or an earlier stage's existence, validation or selection policy. Algorithm 1 processes the stages of one frozen revision in topological order. It retains usable values whose observations are unchanged. Because jobs within a stage are independent, the inner loop over a stage's jobs may run in parallel. A job that is still unusable after the loop is classified as failed, either because it exhausted its budget or because its observation is unavailable. Dependents of failed prerequisites are classified as *blocked* without calling an agent. Blocked is an explicit terminal verdict, not a satisfied requirement.

Theorem 8 (Terminal outcome)**.** *Under A0–A6 and Algorithm 1, once external input and configuration stabilize, the controller eventually reports either a publishable snapshot or an explicit failure, blockage or global conflict. A new topological pass dispatches at most $k|\mathcal{J}_*|$ requests, where $\mathcal{J}_*$ contains all job instances appearing during the pass. Publication follows if every required observation is available, every dirty job obtains a committed proposal within its budget, and the resulting snapshot satisfies G.*

Proof. At the first stage external inputs are stable, so normalization produces finitely many jobs. A usable job needs

no request. Every other available job either succeeds or exhausts its finite budget, because the controller progresses fairly and every dispatch eventually resolves or times out (A6); unavailable jobs are classified directly. Inductively, the predecessors of each later stage are stable. If one has failed, the stage is blocked; otherwise pure normalization and observations yield a fixed finite job set with fixed signatures, and the same argument applies. Independence within a stage ensures that a sibling cannot change a job's observation. With finitely many stages the global predicate is eventually evaluated. Summing at most k dispatches per instantiated job gives the request bound, and the additional hypotheses exclude every failure case. □

Finite budgets guarantee termination, not successful synthesis. Cyclic dependencies lie outside A0: a Boolean value updated by $v \leftarrow \neg v$ along a feedback edge oscillates forever even though each proposal is locally valid. Convergence for cyclic consistency restoration requires an independent well-founded ranking [19], [20].

5.6 Safe Reconfiguration (R6)

Adding an agent, changing a rule, revoking a permission or replacing a validator is a configuration transformation $h : C \mapsto C'$. Adding a security reviewer, for instance, creates new targets and obligations and extends G. Keeping the earlier publication as "current" while the reviewer has produced nothing would be incorrect. This is the generative-workflow analogue of safe dynamic change in component systems [21], [22].

Theorem 9 (Transactional reconfiguration)**.** *Let S satisfy the invariant of Theorem 2. A configuration transaction that checks A0–A4 for C', invalidates every changed observation or authority, reconciles structure and jobs, and recomputes completion yields a state that satisfies the invariant under C'. If it also preserves A5 and configuration changes eventually stop, Theorem 8 applies to the new revision.*

Proof. The installed structure is in the range of $N_{C'}$, which gives (i). A retained usable record has the same effective observation and compatible authority, so its evidence still proves (ii)–(iii); every other record is historical or invalidated and cannot pass the usability gate. Requests whose job, contract or authority changed are cancelled, and unchanged ones must still pass the current commit guard, which gives (iv). No historical publication is relabeled current, and any new publication must pass the recomputed predicate, which gives (v). Atomic installation prevents readers from observing mixed configurations. □

Revocation cancels future commitment but cannot recall data already sent to an agent. Rollback and compensation of external side effects need additional protocols.

5.7 Extensions

Structured ingress. Natural-language input can enter through a separate *lifting* agent that proposes a typed source delta Δ. Treating Δ as an untrusted Edit, checked for ownership, types, source invariants and the absence of writes into controller-owned namespaces, preserves Theorem 2 unchanged (scenario S27). Fidelity of Δ to the user's intent needs an explicit relation and a checker or human confirmation.

Shared writers. A shared-context update is an authorized Edit with an expected-version check: of two writers starting from version v, only the first to commit succeeds (scenario S24). This prevents lost updates but does not resolve semantic disagreement.

External observation. Let W hold an outbox and a remote observer's projection. Committed events may be copied into W; loss, duplication and delivery affect only W. Admission guards and evidence never read W, and a failed copy never blocks the controller.

Proposition 10 (Observational extension)**.** *Erasing W maps every run of the extended system to a run of the controller, allowing stutters, so observer transport cannot falsify the controller's safety invariant.*

Proof. An extended controller step has the base step's guard and authoritative effect plus an optional copy into W, so it projects to that base step. A pure transport step changes only W and projects to a stutter. Induction transfers safety. □

TABLE 4
Counterexamples, the discipline that excludes them, and where the evaluation exercises them (M: TLC mutant; Ab: implementation ablation; S: scenario)

Counterexample	Required discipline	Exercised
Clause changes, interface signature equal	Adequate footprint (A7)	S03, S17
Reply arrives after a source edit	Cancellation or atomic recheck	M, Ab, S11
Two locally valid outputs disagree	Global predicate G	Ab, S18
Exhausted request retried every pass	Persistent per-episode budget	M, Ab, S14
New matching context item added	Membership in footprint	M, Ab, S09
Prompt or validator changes, input does not	Contract η in signature	M, Ab, S23
Revoked owner's reply arrives	Authority in guard	M, Ab, S22
Locally valid value breaks structure	Candidate normalization	Ab, S19
Value read while its prerequisite is stale	Upstream gate in U_j	Ab, S15
Feedback edge flips a value forever	Acyclic schedule (A5)	Theorem 8
Old request arrives after recreation	Never-reused request identities	Test T02

5.8 Boundary Cases and Summary

Table 4 lists counterexamples that delimit the results. Each identifies a specification or implementation obligation; the last column shows where the evaluation exercises it. Table 5 maps every guarantee to its assumptions and its evidence.

6 Instantiation

6.1 Document-Service Case Study

The executable case study instantiates the running example as a ranked chain of four views per source document: requirements, architecture, implementation and verification.

TABLE 5
Guarantees, assumptions and evidence. ●: complete for the stated scope; ○: partial or scoped (e.g., a lemma used by the proof, a bounded instance); –: none. *Exec.* denotes executable checks on the reference controller (Section 7).

Guarantee	Requirement	Assumptions	Paper proof	TLAPS	TLC	Exec.	Remaining obligation
Thm. 1 Structural kernel	R1	–	●	○	–	●	Connect typed kernel to the full protocol model
Thm. 2 Controller safety	R1–R3	A0–A4	●	○	○	●	Mechanize the full inductive invariant
Cor. 3 Domain correctness	R2	A7	●	○	–	○	Adequacy and soundness beyond the finite language
Thm. 4 Forward simulation	R2	A0–A4	●	–	–	○	Checked refinement mapping to the atomic model
Thm. 5 Exact invalidation	R4	A2	●	–	–	●	Machine proof
Prop. 6 Isolation	R4	A2	●	–	–	○	–
Thm. 7 Generation bound	R5	A3	●	○	○	●	History-counting induction across episodes
Thm. 8 Terminal outcome	R5	A0–A6	●	–	○	○	Unbounded temporal mechanization
Thm. 9 Reconfiguration	R6	A0–A5	●	–	○	●	Parameterized preservation proof
Prop. 10 Observation	R3	A3	●	–	–	○	No completeness/authenticity of remote timelines
Thm. 11 Policy table	R2	A7	●	○	–	●	Refinement of the Python interpreter

TABLE 6
Case-study bindings, their observations and validator strength

View	Observation (inputs)	Local validator
Requirements	Source policy	Clause IDs; exact Boolean policy
Architecture	Usable requirements	Operation list; typed ID choice (string or integer)
Implementation	Interface, clause, selected findings	*Sound and complete*: exact six-row policy table (Thm. 11)
Verification	Table, clause, selected findings	Coverage and typed ID metadata
Security (added)	Verification, findings	Operation-assessment coverage

A fifth security-review view can be added at run time. Each view has one stable target and one generated JSON attribute per document (Table 6). The three requirements are as follows. R01 requires stable string document identities. R02 specifies the exact lifecycle behavior (state and status). R03 requires complete view metadata and matching identifier types in architecture and verification.

Two choices make the case study deliberately adversarial. First, the architecture validator *locally* accepts either a string or an integer identifier type, but the downstream structural expression requires strings. An integer proposal is therefore locally valid yet must be rejected by candidate normalization (failure F2 at the structural level). Second, verification may also choose an integer type independently, which only the global predicate catches (F2 at the publication level).

6.2 A Behavioral Validator with a Proof

Coordination invariants are useful even with weak validators, but stronger domain claims need a sound validator (Corollary 3). The implementation view shows how to obtain one by restricting the artifact language. A document has a state in $D = \{\mathsf{open}, \mathsf{archived}\}$ and receives events in $E = \{\mathsf{read}, \mathsf{put}, \mathsf{archive}\}$. A proposal is exactly a total decision table $t : D \times E \to \{200, 409\} \times D$ that maps each state–event pair to (status, next state). A small trusted interpreter executes one row per event and has no other effects. The observation contains a policy bit h; $h = 1$ requires rejecting put on archived documents. The reference behavior is

$$f_h(d, e) = \begin{cases} (200, \mathsf{archived}) & e = \mathsf{archive}, \\ (409, \mathsf{archived}) & h = 1,\ d = \mathsf{archived},\ e = \mathsf{put}, \\ (200, d) & \text{otherwise.} \end{cases} \tag{9}$$

The validator checks the table's exact domain and primitive types and that $t(d, e) = f_h(d, e)$ for all six inputs; it rejects duplicate or missing rows. The footprint contains h, the schema, the interpreter identity and the validator contract.

Theorem 11 (Sound table generation and trace behavior)**.** *For this artifact language the footprint is adequate for extensional equality to f_h, and the validator is sound and complete for it. If a published table has $h = 1$, every finite execution of the interpreter from* open *satisfies: an archived document stays archived; every* put *on an archived document returns 409; and* archive *leads to archived from either state.*

Proof. The domain relation is $R(Z, t) \iff \forall (d, e) \in D \times E.\ t(d, e) = f_{h(Z)}(d, e)$ on a total typed domain. It depends only on h and the fixed schema and interpreter, which gives adequacy. The validator enumerates exactly the six domain elements, so it accepts exactly this relation, which gives soundness and completeness. By Theorem 2, usable tables satisfy the validator. For traces, induct on the length of the event word: each step produces the response and successor of f_1; from archived, read and put stay archived and archive yields archived; put returns 409. □

A change from $h = 0$ to $h = 1$ changes the implementation job's signature although the interface is unchanged (the 409 clause of Section 2). Old pending tables cannot commit, and a current accepted table carries the trace guarantee of Fig. 5. This guarantee comes from restricting the proposal language and trusting a small interpreter. It does not transfer to unrestricted patches that merely implement a similar interface.

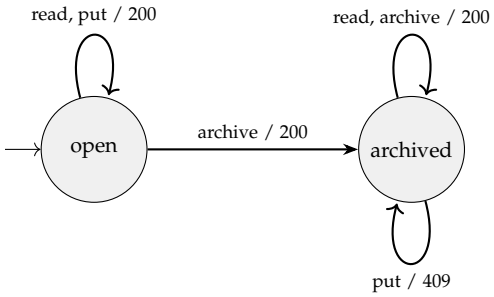


Fig. 5. Checked document policy for $h = 1$ (event / status). Every usable table induces exactly this machine for all event sequences.

6.3 Reference Controller, Formal Model and Proofs

Reference controller. A Python implementation (about 450 lines, `src/vgc`) realizes the transitions of Table 2 for the case study. It serializes public actions with one lock. Every action copies a private candidate, installs effects there, recomputes the complete downstream structure, and installs the candidate only on success. Observations are canonical JSON containing the owner, contract, source clause, usable predecessor values and the full membership of the selected findings. Context version numbers are excluded, because agents do not see them. Request identities (j, e, n) are unique within one controller lifetime. Proposal providers are deterministic or adversarial scripts; no model credentials or remote calls are involved. A separately implemented *trace checker* replays recorded events and independently re-evaluates state continuity, ranked availability, exact observations, dispatch identities and counters, provenance, authority, rollback and publication snapshots.

Formal model. `Core.tla` and `AsyncController.tla` formalize ranked prerequisites, exact observations, historical acceptance, episode counters, request history, suspension, contract and authority changes, and publication evidence in TLA$^+$ [23], [24]. The value relation is abstract: a proposal is valid when it equals the observed source value. `AtomicController.tla` specifies the abstract machine $\mathcal{A}$.

Mechanized lemmas. Four TLAPS [25] modules check individual proof steps. `Budget` bounds the counter within one episode. `ControllerSafety` shows that the commit guard implies a current, authorized, valid commit. `StructuralKernelProofs` establishes coverage, unique producers and reference range under the construction hypotheses of Theorem 1. `DocumentPolicyProofs` establishes validator equivalence, archive absorption and the archived-put conflict. The full inductive invariant of Theorem 2 and the refinement mapping of Theorem 4 are not yet mechanized (Table 5).

7 Evaluation

7.1 Research Questions

The guarantees of Section 5 are proved on paper. The evaluation asks whether they hold in an executable instantiation, whether each mechanism is necessary, and what they buy in practice.

RQ1 *Safety and guard necessity.* Does the controller preserve its safety properties under adversarial asynchronous executions, and does removing any single mechanism break the property it is meant to protect?

RQ2 *Locality and cost.* How much generation does exact dependency tracking avoid compared with coarser invalidation strategies, how does this scale with workspace size, and does it ever change the outcome?

RQ3 *Behavior under change.* Does the reference implementation handle each class of workflow change exactly as the semantics prescribes?

RQ4 *Assurance.* What do the mechanized proofs, bounded model checks and exhaustive checks establish, and what remains open?

7.2 Experimental Setup

Subjects. All experiments use the case study of Section 6.1: the TLA$^+$ model for model checking, and the Python reference controller for everything else.

Why no real LLM? The safety and locality guarantees quantify over *all* proposals; for safety the proposal source is fully nondeterministic. Real model calls would add cost and nondeterminism without exercising behaviors that an adversarial script cannot produce. Our adversarial provider therefore returns valid proposals, locally valid but structurally or jointly inconsistent proposals, malformed values, and plausible answers to a *different* policy question. It also delivers replies late, out of order, duplicated or never. Section 8 discusses what this choice does and does not cover.

RQ1 ablation harness. We drive the reference controller and ten variants, each of which removes exactly one mechanism (two remove both obsolete-reply defenses, as in the formal mutants). Every variant reuses the reference code and overrides only the removed check. Each run starts with one to three documents, budget $k = 2$, and executes 120 randomly chosen actions. The actions are: dispatch; delivery in random order, possibly duplicated; timeout; source edits that flip a policy, add or delete a document; context updates, including equal-content rewrites, unselected findings and availability toggles; contract changes; owner revocation and regrant; adding or removing the reviewer; cooperative repair bursts; and publication attempts. All variants use the same 300 seeds. An *independent oracle* judges the state after every transaction. It recomputes each job's effective observation from the authoritative inputs, remembers the observation each request was dispatched against, and tracks dependency episodes itself; it never reads the controller's signatures. It checks seven properties: P1 no value is exposed as usable whose evidence was produced for a different or unavailable observation (and no request is sent on one); P2 no exposed value is invalid for its current observation; P3 no value is exposed while a prerequisite is unusable; P4 no value from a revoked owner is exposed; P5 no ill-typed structure is installed for an active consumer; P6 no dependency episode receives more than k requests; P7 every published snapshot is fully usable and satisfies G.

RQ2 cost harness. Five strategies share validators, scheduler and guards. They differ *only* in what a job's signature observes: (i) *Exact* (this paper); (ii) *No early cutoff*, where a job depends on the identity of each upstream acceptance rather than its value, as in timestamp-based

build tools; (iii) *Version-keyed context*, where context readers depend on the global context version rather than on the selected content; (iv) *Whole workspace*, where every job sees all documents and the whole shared context, as in shared-message-pool frameworks; (v) *Full regeneration*, where every input change invalidates every job. For each workspace size $N \in \{1, 2, 4, \ldots, 32\}$ we generate 5 seeded streams of 40 changes. Changes are source edits, selected and unselected context changes, equal-content context rewrites, and contract changes of a random stage. After each change we repair to quiescence with a correct provider. We count dispatched requests and the bytes of the observations sent, a proxy for prompt input size. Because the provider is deterministic in the observation, every strategy must accept identical values; we check this after every change.

RQ3 and RQ4. The 28 scenarios and their expectations are fixed in advance in `campaign.json`. Model checking uses TLC [26] with one worker and a 768 MB heap. Proofs run TLAPS with its bundled Z3 4.8.9 in a pinned Linux container, in strict mode and with fingerprints erased. Python 3.13.14 was used throughout.

7.3 RQ1: Safety and Necessity of Each Guard

Bounded model checking. Table 7 reports nine baseline configurations, nine mutants and five nonvacuity witnesses. All baseline configurations satisfy the safety invariants exhaustively, with up to 1,028 distinct states. The frozen-input chain also satisfies eventual terminal classification under weak fairness. Seven of the nine mutants violate their targeted property within two or three actions; for example, removing both obsolete-reply defenses admits "dispatch, edit, obsolete commit". The two survivors each remove only one of the two obsolete-reply defenses, and the remaining defense blocks the stale commit, exactly as predicted in Section 5. The witnesses confirm that acceptance, exhaustion, suspension, publication and contract change are all reachable. The safety results are therefore not vacuous.

Implementation-level ablation. The model-checking mutants use one job and one environmental change. Fig. 6 asks the same question of the real implementation, with multiple documents, concurrent replies and adversarial proposals. Across 300 runs, the reference controller showed **no** violation of any property (0 runs with a violation). It still published in 205 runs (769 publications), so the absence of violations is not due to inactivity. Every non-redundant ablation was detected, each on the property it targets:

- removing both obsolete-reply defenses exposed stale values in 35% of runs, and in 25% values that were *invalid* for the current input: a plausible answer to the old question is not merely outdated but wrong;
- omitting the contract from the signature did so in 80%, and omitting selected context in 43%;
- removing the upstream gate exposed values over stale prerequisites in 99%;
- ignoring authority exposed revoked owners' values in 92%;
- skipping candidate normalization installed ill-typed structure in 44%;
- resetting budgets exceeded k requests per episode in 24%;
- dropping the global predicate published inconsistent snapshots in 34%.

Removing *one* obsolete-reply defense (cancellation or recheck) again produced no violation, which replicates the model-checking result at implementation scale. Several ablations also violate secondary properties. Stale contracts, for example, propagate into P3 and P7, because a stale value feeds its descendants and can reach publication.

TABLE 7
Bounded TLC model checking of the protocol model. J: jobs; k: attempt cap; m: bound on environmental changes; *Distinct*/*Total*: distinct and generated states; *Path*: actions in the shortest counterexample or witness. All 23 runs met their expectation (108 s in total, one worker).

Configuration	J	k	m	Distinct	Total	Path	Result
Baseline protocol (safety invariants; liveness where stated)							
One job	1	1	1	87	316	–	Pass
One job	1	2	1	227	1,061	–	Pass
Two-job chain	2	1	1	292	1,307	–	Pass
Two-job chain, frozen	2	2	0	23	107	–	Pass
Fan-out, frozen	3	1	0	20	95	–	Pass
Four-stage workflow, frozen	4	1	0	14	63	–	Pass
Observation reversion	1	1	2	1,028	4,430	–	Pass
Reordered delivery	2	1	1	475	2,345	–	Pass
Chain, liveness	2	2	0	23	107	–	Pass
Mutants (one guard removed; target property in Table 4)							
Cancellation removed	1	1	1	93	328	–	Survives[a]
Commit recheck removed	1	1	1	87	358	–	Survives[a]
Both obsolete defences removed	1	1	1	31	69	3	Violated
Exhausted budget reset	1	1	1	45	89	3	Violated
Suspension ignored	1	1	1	14	20	2	Violated
Authority ignored	1	1	1	32	98	3	Violated
Revision check removed	1	1	1	14	21	2	Violated
Contract omitted	1	1	1	33	60	3	Violated
Selected context omitted	1	1	1	32	57	3	Violated
Nonvacuity witnesses (negated reachability goal)							
Acceptance	1	1	1	21	32	2	Reached
Budget exhaustion	1	1	1	22	33	2	Reached
Suspension	1	1	1	6	7	1	Reached
Publication (empty workflow)	1	1	1	14	21	2	Reached
Contract change	1	1	1	5	6	1	Reached

[a]Survives because the other obsolete-response defence still blocks the stale commit; see RQ1.

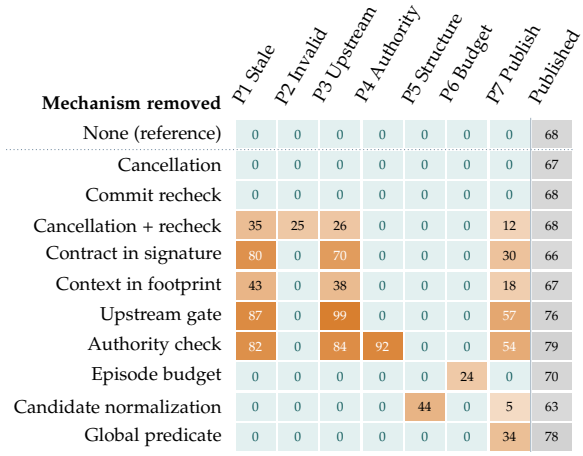


Fig. 6. RQ1: implementation-level guard ablation under 300 seeded adversarial executions per variant (120 steps each). Cells give the percentage of runs in which the independent oracle observed at least one violation of property P1–P7 (Section 7.3); the last column gives the percentage of runs that published at least once. The reference controller (top row) shows no violation. Removing either obsolete-response defence alone is masked by the other.

A defect found by the campaign. The randomized campaign also exposed a robustness defect in the original reference implementation. `edit_source` and `reconfigure` record a rejection when candidate normalization fails, but `update_context` let the exception escape. The path is as follows. Context becomes unavailable, so the implementation stage is suspended. An integer-identifier architecture is then committed, which is admissible because no active consumer exists. Re-enabling the context fails normalization. With the same seeds, the unfixed controller raised this exception in 7 of 300 reference runs (10 exceptions in total). The state remained atomic, because the failure occurred on the private candidate, and no safety property was violated (0 runs with a violation), but the API broke its contract. We fixed the defect (the action now records the rejection) and added a regression test. The underlying semantic subtlety is now listed as a limitation (Section 10). We also found that the original candidate constructor instantiated the base class, which silently bypassed overridden checks in subclasses; it now preserves the subclass type. The numbers above are from the fixed implementation.

Answer to RQ1. No safety violation was observed in 9 exhaustive model-checking configurations or in 300 randomized adversarial executions of the reference controller. Each of the 8 ablations that remove a non-redundant mechanism violated its targeted property, both in the model (where a corresponding mutant exists) and in the implementation. The cancellation and recheck defenses are individually redundant but jointly necessary.

7.4 RQ2: Locality and Cost

Fig. 7 plots generation requests and observation bytes per change against workspace size, and Table 8 breaks requests down by change type for $N = 32$. With exact signatures, cost grows only with the number of jobs a change actually affects. A source edit regenerates that document's four jobs. A selected-context change regenerates the two jobs that read the finding. Unselected changes and equal-content rewrites regenerate nothing. Only contract changes touch every job of the changed stage, because every such job's contract changed. Full regeneration and the whole-workspace footprint instead scale linearly with N. At $N = 32$, exact signatures needed 10.8 requests per change on average, versus 128.0 for full regeneration: a 11.9× reduction. The reduction is 8.1× against the whole-workspace footprint, 3.0× against version-keyed context, and 2.3× against the strategy without early cutoff.

The savings in transmitted observation size are larger still, because a whole-workspace observation also grows with N. Per change, the whole-workspace strategy sent 52× the bytes of the exact strategy, and full regeneration 9×. Exact observations are small and their size is independent of N. In every run, all five strategies accepted identical values after every change (outcome equivalence: all runs). Coarser footprints are thus sound but wasteful, and the savings never come at the price of a different result.

The scenario campaign confirms the mechanism on individual interventions. A repair pass with no relevant change (S02), an unselected context update (S08) and a new context version with equal selected content (S10) caused no request at all. An architecture contract change repaired to an equal output caused exactly one request, because early cutoff restored the descendants from cache (S16). A tightened clause propagated to all four views, because their observations changed despite the unchanged interface (S17).

TABLE 8
RQ2: mean generation requests per change by change type for $N = 32$ documents. Exact signatures make the cost of source and context changes independent of N; contract changes necessarily touch every job of the changed stage.

Change type	*Exact (ours)*	*No early cutoff*	*Version-keyed context*	*Whole workspace*	*Full regeneration*
Source clause edit	4.0	4.0	4.0	128.0	128.0
Selected finding changed	2.0	2.0	64.0	128.0	128.0
Unselected finding changed	0.0	0.0	64.0	128.0	128.0
Equal content, new version	0.0	0.0	64.0	0.0	128.0
Contract change	32.0	82.9	32.0	32.0	128.0
All changes	10.8	25.0	32.2	87.7	128.0
Bytes per request	316	235	414	2,027	241

Answer to RQ2. Exact dependency signatures make the generation cost of source and context changes independent of workspace size. At $N = 32$ they reduce requests per change by 11.9× relative to full regeneration and by 2.3–8.1× relative to plausible coarser strategies, with identical outcomes in all runs. These counts are structural: they measure avoided generation calls, not LLM latency or price.

7.5 RQ3: Behavior Under Change

Table 9 groups the 28 scenarios by the class of change they exercise and maps each class to the guarantee it tests. All 28 scenarios met their predefined expectations. Many expectations are refusals: a discarded late reply (S11–S13), an exhausted episode that earns no new budget (S14), a blocked descendant (S15), a joint conflict (S18), a rolled-back candidate (S19), a reviewer whose addition blocks publication until it has assessed every operation (S20), and a rejected publication based on a stale check (S25). Three scenarios published successfully with nonempty generated work (S01, S20 after repair, and S26). The trace checker admitted all 28 recorded traces and rejected all 9 deliberately corrupted ones. The corruptions include acceptance events that hide a source, context or contract change behind a valid value.

Two observations deserve emphasis. First, *reversion does not restore authority*: in S12 the clause changes and then reverts, so the observation bytes return to their original value, but the cancelled request still cannot commit, because the reversion started a new episode. Signatures alone therefore do not give idempotent delivery; request identities are needed as well. Second, *equal content is not a change*: in S10 the shared context is republished at a new version with the same selected findings, and no job is invalidated, because agents never see version numbers (Section 4).

Answer to RQ3. The reference implementation behaved as the semantics prescribes for all ten change classes, includ-

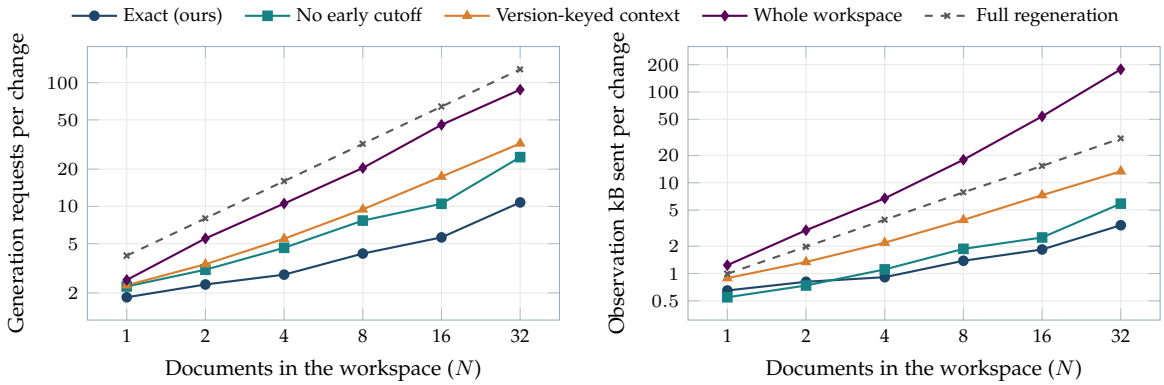


Fig. 7. RQ2: generation work per input change as the workspace grows (log–log; mean over 5 seeds × 40 random changes per size). All strategies share validators, scheduler and guards and differ only in what a job's signature observes; they accepted identical values in every run. Left: dispatched requests. Right: total size of the observations sent to workers, a proxy for prompt input tokens.

TABLE 9
The 28 scripted change scenarios, grouped by the class of change they exercise. Event counts come from the recorded traces: D dispatched requests, A accepted replies, R current rejections or timeouts, X obsolete or duplicate replies discarded, E committed edits/reconfigurations, B refused edits or publications, P publications. A scenario passes when its independently written expectation holds; for several scenarios the expected outcome is a refusal.

Change class	Scenarios (S#)	Guarantee exercised	D	A	R	X	E	B	P	Pass
Initial completion and no-op repair	01, 02	Thm. 2, 5	8	8	0	0	0	0	1	2/2
Source edits, additions, deletions	03, 04, 05, 06, 17	Thm. 1, 5	28	28	0	0	6	0	0	5/5
Shared-context changes and writers	07, 08, 09, 10, 24	Thm. 5, Prop. 6	18	18	0	0	6	1	0	5/5
Obsolete, late, duplicate replies	11, 12, 13	Thm. 2(iv), 4	3	0	1	4	3	0	0	3/3
Budget exhaustion and blocking	14, 15	Thm. 7, 8	4	0	4	0	2	4	0	2/2
Contract change and early cutoff	16, 23	Thm. 5, 9	10	9	0	1	3	0	0	2/2
Joint conflict, rollback, stale check	18, 19, 25	Thm. 2(i),(v)	14	13	1	0	3	2	0	3/3
Reviewer and authority changes	20, 21, 22	Thm. 9	10	9	0	1	4	3	1	3/3
Untrusted ingress, observer loss	26, 27	Prop. 10	4	4	0	0	0	2	1	2/2
Finite policy language (exhaustive)	28	Thm. 11	0	0	0	0	0	0	0	1/1
All scenarios	01–28		99	89	6	6	27	12	3	28/28

ing the refusal cases, and an independent checker confirmed every recorded trace.

7.6 RQ4: Assurance and Remaining Obligations

Table 10 summarizes the mechanized and exhaustive evidence. The four TLAPS modules discharge 25 obligations. The preserved first development run failed three obligations in two modules: proof steps that lacked explicit references to construction and type assumptions. No assumption equivalent to the safety conclusion was added to repair them. For the restricted policy language, exhaustive enumeration of all 8,192 typed tables (both policies) confirmed that the validator accepts exactly one table per policy. All 1,093 event words of length at most six obey the trace properties of Theorem 11. Two clean re-runs from packaged archives reproduced every reported outcome; both predate the defect fix of Section 7.3, after which the scenario, conformance and test stages were re-run.

The evidence is deliberately uneven, and Table 5 makes this explicit. Lemmas used inside the safety and budget proofs are mechanized, but the full inductive invariant, the history-counting argument across episodes, and the refinement mapping to `AtomicController.tla` are proved only on paper. Bounded model checking covers instances with at most four jobs. The implementation-level ablation (RQ1) and the scenario campaign (RQ3) cover larger, concurrent executions of the reference controller, but by testing rather than proof.

Answer to RQ4. The scoped lemmas, bounded model checks and exhaustive checks corroborate the paper proofs at every level we could mechanize. The central parameterized invariant and the refinement mapping remain open mechanization targets.

7.7 Implications for Builders of Agent Frameworks

The results suggest concrete design rules for orchestrators of LLM agents, such as graph-based or conversation-based frameworks.

1) *Key every generated artifact by its exact observation, including the prompt and validator version.* RQ1 shows that omitting the contract or the selected context from the key exposes stale artifacts in a large fraction of runs. RQ2 shows that keying by content rather than by version or timestamp avoids most regeneration.
2) *Give every request an identity, and recheck currentness at commit time.* Either defense alone prevented stale commits in RQ1, but they serve different purposes, so keep both. The commit-time recheck is what makes acceptance safe. Cancellation additionally

TABLE 10
Assurance evidence. For proofs, *Final* counts proved/total obligations in the clean run and *First trial* the preserved failed development run. Executable counts are exhaustive where stated; the other checks cover the recorded executions only.

Artifact	Scope	Final	First trial
Mechanized TLAPS lemmas (strict mode, fingerprints erased)			
`Budget`	Counter bound for one episode	8/8	1 of 7 failed
`ControllerSafety`	Commit guard implies current, authorized, valid commit	2/2	0 of 2 failed
`StructuralKernel`	Kernel coverage, unique producers, reference range	9/9	2 of 6 failed
`DocumentPolicy`	Validator equivalence; archived-put conflict	6/6	0 of 6 failed
Executable checks			
Policy tables	All typed six-row tables, both policies	8,192	exhaustive
Event words	All words of length ≤ 6, strict table	1,093	exhaustive
Trace checker	Recorded scenario traces admitted	28/28	–
Trace checker	Deliberately corrupted traces rejected	9/9	–
Regression tests	Transaction-boundary unit tests	13/13	–
Reproduction	Clean re-runs of packaged archives matched (pre-fix snapshots)	2/2	–

frees the job: without it, an obsolete request stays pending and blocks re-dispatch until its reply arrives or times out.

3) *Separate "accepted" from "usable", and gate consumers on usability.* Removing the upstream gate was the most frequently detected ablation.
4) *Make completion a global predicate, recomputed after every change of team or rules.* Locally valid outputs that disagree reach publication otherwise.
5) *Budget per dependency episode, not per scheduler pass.* Per-pass budgets let a failing request be retried indefinitely.

None of these rules depends on how good the underlying model is.

8 Threats to Validity

Construct validity. Our safety properties P1–P7 operationalize Theorem 2 and Theorem 7. The oracle recomputes observations from the specification, but it shares the case study's definitions of footprint and validator with the controller. A shared misunderstanding of those definitions would go undetected. Observation bytes are only a proxy for prompt tokens, and request counts measure avoided calls, not latency or monetary cost.

Internal validity. Each ablation reuses the reference code and overrides only the removed check, which limits confounding. We found and fixed one harness problem during development, a base-class candidate constructor that silently bypassed overrides; all reported numbers come from the corrected harness. Scenario expectations were written by the author before execution. They are fixed in the repository, but they are not independent of the designer's understanding of the semantics. The trace checker trusts recorded events and cannot detect an omitted final suffix.

External validity. We used scripted and adversarial providers instead of real LLMs. This is sound for safety, because the semantics quantifies over all proposals, and for structural cost, because request counts do not depend on answer quality. It does not tell us how often real models produce invalid or conflicting proposals, or how much wall-clock time the savings represent. The case study is small (four or five views per document, a six-row behavioral language), and bounded model checking covers at most four jobs. Larger artifact languages, such as full source files with test-suite validators, make footprint adequacy and validator soundness harder to establish (Example 1).

Conclusion validity. The runs are deterministic functions of their seeds and are not independent samples of a deployment population. We therefore report counts and proportions, not significance tests. All seeds, configurations and source hashes are recorded with the results.

9 Related Work

LLM-based multi-agent software engineering. MetaGPT [2] encodes standard operating procedures as role prompts and structured hand-offs. ChatDev [3] organizes agents into a chat chain, and AutoGen [4] provides programmable multi-agent conversation. SWE-agent [5] studies agent–computer interfaces for repository-level repair, evaluated on SWE-bench [9]. Surveys map this landscape [1], [6], [7]. Cemri et al. [13] analyze failures of multi-agent LLM systems and report specification issues, inter-agent misalignment and task-verification failures as major categories. These works orchestrate agents by conversation or fixed pipelines. Our work is complementary: it specifies when an agent's output may be accepted, reused and released, and it proves properties that hold for any agent behavior. The rules of Section 7.7 could be adopted by these frameworks.

Validating individual LLM outputs. Hallucination [27] motivates validation of generated artifacts. Rigorous test-based evaluation reveals many incorrect programs that pass weaker tests [8]. Constrained decoding enforces syntactic and semantic constraints during generation [10], [11], and guardrail toolkits filter inputs and outputs [12]. All of these act on a single response. Our validators play the same role, but they are evaluated against the *recorded* observation and combined with currentness, authority, structural and global checks. A response that passes every per-response check can still be stale, unauthorized or jointly inconsistent.

Runtime enforcement and shields. Schneider characterizes the policies enforceable by execution monitoring [28]. Runtime verification and enforcement study monitors that observe or modify executions [29], [30]. Shield synthesis corrects reactive outputs to satisfy safety specifications [31]. Our controller also excludes invalid steps. Its state, however, includes persistent generated artifacts, dependency episodes and transactional configuration, and it defers or rejects proposals instead of correcting them.

Consistency management and model transformation. Instant consistency checking [32] and inconsistency tracking in design models [33] incrementally re-evaluate consistency rules after model changes, based on the model elements each

rule accessed. Our footprints play the role of these scopes, extended with selected context and generation contracts. ATL [14] provides typed transformation rules. Bidirectional transformations [34] and networks of models [19], [20] study consistency restoration across views and show that pairwise consistency does not imply global consistency. Stevens connects software build with megamodel consistency, including soundness and incrementality [35]. Our setting uses directed, single-producer stages with nondeterministic primitive proposals, and adds rejected values, obsolete replies and contract changes.

Incremental computation and build systems. Self-adjusting computation [36], build-system theory [37] and sound incremental builds with dynamic dependencies [38] provide early cutoff and minimal rebuilding for *deterministic* tasks. Our signatures are rebuild keys, and failed-attempt memory adds an explicit terminal unsuccessful outcome. Unlike deterministic rebuilding, two valid repairs may produce different values, so we preserve admissibility relative to observations rather than equality with a fresh run.

Dynamic reconfiguration. Kramer and Magee's quiescence [21] and Vandewoude et al.'s tranquility [22] define when components can be safely replaced at run time. Theorem 9 plays this role for agents, rules and validators: changes are atomic configuration transactions that invalidate exactly the affected evidence and recompute completion.

Specification and verification. We use TLA$^+$ [23], [24] with TLC [26] and TLAPS [25], as practiced in industry [39]. Our refinement argument follows the forward-simulation method [17], [18]. IronFleet [40] shows how such refinements can be mechanized down to implementations, which is our main open obligation. Our ablation study follows the logic of mutation analysis [41], applied to guards instead of program statements.

10 Limitations and Future Work

Trusted computing base. The guarantees assume correct normalization, signature construction, authorization, transaction isolation, validator execution and publication. A concrete implementation should be shown to refine the transitions of Table 2, including durable request identities and crash recovery. The reference controller keeps identities only for one process lifetime.

Footprint adequacy and validator strength. These are the principal semantic boundary. Engineers must show that observations include all information the desired relation needs, including negative dependencies and shared context. Parsers and finite test suites usually establish weaker relations. Theorem 11 discharges the obligation for one finite language. Proof-carrying proposals checked by a trusted verifier are a promising route to richer languages.

Deferred structural checks. Structural constraints bind only active consumers. A value admitted while its consumer is suspended can therefore make every later reactivation inadmissible, until the value itself is repaired. This is the path behind the defect found in RQ1. The semantics handles it safely by rejecting the reactivating edit, but it is a liveness hazard. Type-checking configured rather than enabled structural expressions, or invalidating the offending upstream value when reactivation fails, would remove it.

Acyclicity and effects. The acyclic discipline excludes unrestricted collaborative editing and cyclic consistency restoration. Generated code is treated as inert data. An agent with independent file-system, network or deployment permissions can cause effects outside the machine, which require mediated tool actions and a larger transition system.

Mechanization. Mechanizing the full inductive invariant, the history-counting induction and the refinement mapping, and connecting the structural kernel to the protocol model, are the main remaining verification tasks. A study with real LLM agents on larger artifacts, measuring failure frequencies and wall-clock cost, is the main remaining empirical task.

11 Conclusion

Reliable collaboration among generative agents is a coordination problem as much as a generation problem. Once agents are treated as untrusted proposers and a deterministic controller owns structure, dependencies, acceptance and publication, useful guarantees become provable without assuming anything about the agents. Exact dependency signatures and atomic commitment exclude stale, unauthorized and inconsistent results. A publication barrier separates local acceptance from global readiness. Per-episode budgets and an acyclic schedule yield an honest success-or-failure verdict, and transactional reconfiguration extends these properties to changes of team and rules. The evaluation shows that each mechanism is necessary and that exactness pays off: on the largest workspaces, most generation calls of coarser strategies were unnecessary.

Data Availability

The replication package contains the TLA$^+$ specifications and proofs, the Python reference controller, the scenario definitions and traces, the RQ1/RQ2 campaign (`experiments/tse_campaign.py`), and the scripts that generate every table, plot and number in this paper (`experiments/tse_paper.py`, `experiments/analysis.py`), together with raw logs, exact commands, tool versions and source hashes. `python scripts/reproduce.py all --profile core` reruns the formal and scenario stages and deliberately reports the open central proof gate as incomplete; `python scripts/reproduce.py tse-campaign` and `tse-paper` re-run the RQ1/RQ2 campaign and regenerate this paper's evaluation fragments. [Archive DOI to be inserted upon acceptance.]

Declaration on the Use of AI

Generative AI assisted in developing the formalization, the artifact, the prose and internal critical reviews. These reviews are not independent human peer review. The author checked and is responsible for all claims, proofs, references and the final text.

REFERENCES

[1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[3] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 174–15 186.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[6] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[7] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[8] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[10] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.

[11] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv:2307.09702, 2023.

[12] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023, pp. 431–445.

[13] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.

[14] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, "ATL: A model transformation tool," *Science of Computer Programming*, vol. 72, no. 1–2, pp. 31–39, 2008.

[15] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990.

[16] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[17] N. Lynch and F. Vaandrager, "Forward and backward simulations: Part I: Untimed systems," *Information and Computation*, vol. 121, no. 2, pp. 214–233, 1995.

[18] M. Abadi and L. Lamport, "The existence of refinement mappings," *Theoretical Computer Science*, vol. 82, no. 2, pp. 253–284, 1991.

[19] H. Klare and J. Gleitze, "Termination and expressiveness of execution strategies for networks of bidirectional model transformations," *Formal Aspects of Computing*, vol. 35, no. 3, pp. 1–35, 2023, article 15.

[20] P. Stevens, "Maintaining consistency in networks of models: bidirectional transformations in the large," *Software and Systems Modeling*, vol. 19, no. 1, pp. 39–65, 2020.

[21] J. Kramer and J. Magee, "The evolving philosophers problem: Dynamic change management," *IEEE Transactions on Software Engineering*, vol. 16, no. 11, pp. 1293–1306, 1990.

[22] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D'Hondt, "Tranquility: A low disruptive alternative to quiescence for ensuring safe dynamic updates," *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 856–868, 2007.

[23] L. Lamport, "The temporal logic of actions," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 3, pp. 872–923, 1994.

[24] ——, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[25] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto, "TLA+ proofs," in *FM 2012: Formal Methods*, ser. Lecture Notes in Computer Science, vol. 7436. Springer, 2012, pp. 147–154.

[26] Y. Yu, P. Manolios, and L. Lamport, "Model checking TLA+ specifications," in *Correct Hardware Design and Verification Methods (CHARME)*, ser. Lecture Notes in Computer Science, vol. 1703. Springer, 1999, pp. 54–66.

[27] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[28] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[29] M. Leucker and C. Schallhart, "A brief account of runtime verification," *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[30] S. Pinisetty, Y. Falcone, T. Jéron, H. Marchand, A. Rollet, and O. Nguena Timo, "Runtime enforcement of timed properties revisited," *Formal Methods in System Design*, vol. 45, no. 3, pp. 381–422, 2014.

[31] B. Könighofer, M. Alshiekh, R. Bloem, L. Humphrey, R. Könighofer, U. Topcu, and C. Wang, "Shield synthesis," *Formal Methods in System Design*, vol. 51, no. 2, pp. 332–361, 2017.

[32] A. Egyed, "Instant consistency checking for the UML," in *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, 2006, pp. 381–390.

[33] ——, "Automatically detecting and tracking inconsistencies in software design models," *IEEE Transactions on Software Engineering*, vol. 37, no. 2, pp. 188–204, 2011.

[34] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, and A. Schmitt, "Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem," *ACM Transactions on Programming Languages and Systems*, vol. 29, no. 3, p. 17, 2007.

[35] P. Stevens, "Connecting software build with maintaining consistency between models: towards sound, optimal, and flexible building from megamodels," *Software and Systems Modeling*, vol. 19, no. 4, pp. 935–958, 2020.

[36] U. A. Acar, M. Blume, and J. Donham, "A consistent semantics of self-adjusting computation," in *Programming Languages and Systems (ESOP 2007)*, ser. Lecture Notes in Computer Science, vol. 4421. Berlin, Heidelberg: Springer, 2007, pp. 458–474.

[37] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, pp. 1–29, 2018, article 79.

[38] S. Erdweg, M. Lichter, and M. Weiel, "A sound and optimal incremental build system with dynamic dependencies," in *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015, pp. 89–106.

[39] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, "How Amazon web services uses formal methods," *Communications of the ACM*, vol. 58, no. 4, pp. 66–73, 2015.

[40] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "IronFleet: Proving practical distributed systems correct," in *Proceedings of the 25th Symposium on Operating Systems Principles*. New York, NY, USA: ACM, 2015, pp. 1–17.

[41] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.